Dipartimento di Elettronica e Informazione
Politecnico di Milano
prof. ***Carlo Ghezzi***
prof. ***Angelo Morzenti***
prof. ***Pierluigi San Pietro***
20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (39) 02-2399.3400
Fax (39) 02-2399.3411

Recupero di Ingegneria del Software –

15 luglio 2002

Cognome

Nome

Matricola

Sezione (segnarne una) Ghezzi ☐, Morzenti ☐, SanPietro ☐

Recupero prova: 1 2

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno presi in considerazione.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, ecc.).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. Tempo a disposizione: 1h.30m se si svolge una sola parte, 3h per entrambe le parti.
6. Punteggio totale a disposizione per ciascuna prova: 13 punti. Una prova può essere sommata al risultato dell'altra prova (già svolta o quella presenbte) solo ottenendo almeno 6 punti.

PROVA 1: Esercizio 1.1 (5 punti)
Parte (a)
Si considerino le seguenti classi Java.
Completare opportunamente le parti con i puntini, aggiungendo, ove necessario, modificatori di visibilità, dichiarazioni e/o implementazioni di metodi, ecc. Si trascuri l'implementazione di altri metodi oltre a quelli strettamente necessari. Non implementare equals, repOk, toString, ecc.

```
package Linker;
public abstract..... class Modulo{
//OVERVIEW: Tipo mutabile che rappresenta una unità di codice scritto in un
//linguaggio specificato.
        private String linguaggio; //memorizza il nome del linguaggio
        public abstract String getCodice()
        //EFFECTS: restituisce una rappresentazione stampabile del codice
}
        public Modulo(String linguaggio) {
        //EFFECTS: inizializza this a essere un modulo scritto nel linguaggio dato
                this.linguaggio=linguaggio;
        }
        public String getLinguaggio() {
        //EFFECTS: restituisce il nome del linguaggio in cui e' scritto this
        return linguaggio;
}


package CoseUtili;
public class CodiceScorrettoException extends Exception {
//da non implementare}

package CoseUtili;
public class MetodiUtili {
        public static String ByteCode2Obj (String codiceByteCode)
        throws CodiceScorrettoException {
        //REQUIRES: codiceByteCode è una stringa contenente il codice corretto di
        //un modulo di codice ByteCode
/       /EFFECTS:restituisce la traduzione di  codiceByteCode in codice oggetto OBJ
        //NB: da non implementare!!!
        }
}

package Linker;
 public..... class ModuloByteCode extends Modulo{
//OVERVIEW: Tipo che rappresenta un modulo di codice in
//linguaggio JavaByteCode
//AF: una coppia (this.linguaggio, this.codice)
//rep: una stringa
protected..... String codice;  //anche private è corretto
        public ModuloByteCode (String codice) {
        //inizializza this a contenere il codice, scritto in linguaggio "JavaByteCode"
                super("JavaByteCode");
                this.codice=codice;.....
        }

        public String getCodice() {
                return this.codice.....
        }
        public ModuloObj traduciInObj()
                ..... throws CodiceScorrettoException {
                return new ModuloObj(CoseUtili.MetodiUtili.ByteCode2Obj( this.codice);.....
        }
```

```
public..... class ModuloObj extends .....Modulo{
//OVERVIEW: Tipo che rappresenta un modulo di codice in
//linguaggio oggetto (OBJ)
//rep: una stringa
      protected String codice;.....
      public ModuloOBJ (String codice) {
            super("OBJ");
            this.codice=codice;.....
      }
      public String getCodice() {
            return this.codice
      }
}
```

Esercizio 1, Parte (b)

In un file separato di nome LinkerUser.java, contenuto in un altro package, è riportata la seguente classe (NB: si rammenta che import permette di importare una classe e utilizzarne i metodi pubblici senza dot notation):

```
import Linker.*;
public class LinkerUser {
  public static void main(String args[]) {
      Modulo m[] = new Modulo[4];
      String cod1;
      .....//qui cod1  assegnato a codice JavaByteCode corretto;
      try {
            m[0] = new ModuloByteCode(cod1);
            m[1] = m[0].traduciInObj();
            m[2] = new ModuloObj(m[1].getCodice());
            stampa(m);
      }
      catch (Exception e) { System.out.println("Eccezione");}
}
public static void stampa(Modulo moduli[]) {
      for (i=0; i<moduli.length; i++)
            System.out.println(moduli[i].getLinguaggio());
}
```

Si ricorda che il metodo statico System.out.println() stampa in ouput il valore dell'espressione passata come argomento.

Riportare in modo preciso il risultato dell'esecuzione (che cosa viene stampato, eventuali errori o eccezioni run-time): Per ogni chiamata del metodo getLinguaggio() indicare qual è la classe in cui è definito il metodo effettivamente chiamato durante l'esecuzione. Se viene sollevata un'eccezione, dire quale eccezione viene sollevata, da chi viene intercettata e come prosegue, se prosegue, l'esecuzione.).

La riga m[0].traduciInObj(); richiederebbe un cast a ModuloByteCode e quindi così il prgoramma non compila.
Se ci fosse il cast il risultato sarebbe:
stampa:
JavaByteCode, OBJ, OBJ, Eccezione
Genera NullPointerException quando cerca di stampare m[3]: l' eccezione è unchecked e si propaga al chiamante, che la intercetta nel catch.
(NB: entrambe le soluzioni, con o senza cast, sono accettabili).

Cosa succederebbe se si eliminasse il blocco try...catch dal main (lasciando tutte le altre istruzioni all' interno del blocco)?

il programma non compila, perche' la traduciInObj dichiara throws di eccezione checked

PROVA 1: Esercizio 1. 2 (8 punti)
Si consideri la seguente specifica della classe MultiSet:

```
public class MultiSet {
    // OVERVIEW: multi-insiemi (detti anche bag) di oggetti di tipo Object:
    //un multiinsieme è un contenitore di elementi in ordine qualunque, di molteplicità qualunque.
    //Il valore null non fa parte di un mulltiinsieme
    // Usa equals per confrontare gli oggetti
    //Un tipico MultiSet è una sequenza s = x1 x2... xn, dove
    //ciascun valore xi compare in s un numero >=1 di volte.
    //Per es.: A B A C A C, dove A, B, C sono valori di tipo Object.
    //costruttori:
    public MultiSet(){
    //EFFECTS: inizializza this come insieme vuoto
    }
    //metodi:
    public MultiSet insert(Object x);
        //REQUIRES: x!= null;
        //MODIFIES: this
        //EFFECTS: aggiunge x in una posizione qualunque di this e ritorna this
    public void remove(Object x);
        //REQUIRES: x!= null;
        //MODIFIES: this
        //EFFECTS: se x è in this, ne elimina una qualunque (e una sola) occorrenza da this
    public int molt(Object x)
        //REQUIRES: x!= null;
        //EFFECTS: restituisce il numero di elementi E di this per i quali x.equals(E) è true
    }
```

a) Sono dati i seguenti metodi:

```
public static boolean condizione1(MultiSet m, Object z) {
 //REQUIRES: m<> null, z<> null
//EFFECTS: restituisce .....…..?
  int i = m.molt(z);
        int i = m.molt(z); m.insert(z); //NB: errore in originale
  return (i==j-1);
}
public static boolean condizione2(MultiSet m, Object z) {
 //REQUIRES: m<> null, z<> null
//EFFECTS: restituisce .....……?
  m.insert(z); //NB: errore in originale
  int j = m.molt(z);
  int i = m.molt(z);
  return (i==j-1);
}
```

calcolare qual è l'effetto della chiamata dei due metodi (cioè valore ritornato e side effects), per ogni oggetto z di tipo Object e per ogni MultiSet m, sotto l' ipotesi che l'implementazione di MultiSet sia corretta.

Entrambi i metodi inseriscono m in z, ma il primo restituisce sempre true, il secondo sempre false,

b) Completare, con AF e RI e il codice del metodo molt, la successiva implementazione della classe MultiSet, che utilizza la seguente classe Set (di cui è assegnata solo la specifica).

```
public class Set {
    // OVERVIEW: insiemi di oggetti di tipo Object. Il valore null non fa parte dell'
    // insieme :per es.: {A,B, C } dove A, B, C sono valori di tipo Object.
    // Usa equals per confrontare gli oggetti e non introdurre duplicati.
    //costruttori:
    public Set(); //EFFECTS: inizializza this come insieme vuoto
```

```
        //metodi:
    public void insert(Object x);
        //MODIFIES: this
        //EFFECTS: this_post = this + {x}
    public void remove(Object x);
        //MODIFIES: this
        //EFFECTS: this_post = this – {x}
    public boolean isIn (Object x)
        //EFFECTS: restituisce true  sse x.equals elemento in this
    public Iterator elements()
        //EFFECTS: restituisce un iteratore a tutti gli elementi di this, in un ordine qualunque
        //REQUIRES: this non cambia mentre l'iteratore è in uso
    }

public class MultiSet {
    //(scrivere qui la funzione di astrazione, definita in base al valore tipico descritto nella specifica, e
    preceduta da eventuali definizioni utili:)
    .............
    .............
    .............
    /Sia c un generico oggetto concreto, con c.ins={c1, .., cn}, dove ci e' una Coppia
    // Allora:
    //AF(c)= c1.obj..... c1.obj  c2.obj ..... c2.obj ..... cn.obj ..... cn.obj
    //          |----------------------|  |----------------------|       |----------------------|
    //          c1.i volte                  c2.i volte                  cn.i volte
    //Una definizione più formale è invece la seguente:
    //Definizioni: z^n sia z z ...z (z concatenato n volte) per ogni Object z; .....
    // per ogni c oggetto concreto Multiset e x oggetto di tipo Coppia, .....
    // c' = c – x sia definito come: c'.ins=c.ins.remove(x),      .....
    // per ogni oggetto concreto c, detto x un qualunque valore in c.ins,       .....
    //AF(c)= (x.obj)^(x.i) AF(c - x)

            .....
    //rep:
    protected Set ins;
    //RI=c.ins<>null &&.....
        c.ins contiene solo oggetti di tipo Coppia &&.....
        forall x in c.ins, x.i>0.....
        forall x!=y in c.ins, if c.ins.isIn(x) && c.ins.isIn(y) ==> x.obj!=y.obj&& .....
        .....
    private class Coppia {
        /OVERVIEW: Tipo immutabile rappresentante una coppia (Object,int)
        //rep di Coppia:
        Object obj;
        int i;
        Coppia(Object o, int m) {obj=o; i= m;}
        .....//qui definire altri metodi se ritenuti necessari
        .....occorre ridefinire equals:
        public boolean equals(Object z) {
            if (!(z instanceof Coppia)) return false;
            return equals((Coppia) z); }
        }
        public boolean equals(Coppia c) { return (obj== c.obj && i == c.i);}
        .....
    }
    public MultiSet(){ins = new Set();}
    //metodi:
    public MultiSet insert(Object x){
        for (Iterator itr =ins.elements(); itr.hasNext ();) {
            Coppia c = (Coppia) itr.next();
```

```
                if (c.obj.equals(x)) {
                    ins.remove(c);
                    ins.insert(new Coppia(c.obj,c.i+1));
                    return this;
                }
            }
            ins.insert(new Coppia(x,1)); return this;
        }
        public void remove(Object x){
            for (Iterator itr =ins.elements(); itr.hasNext ();) {
                Coppia c = (Coppia) itr.next();
                if (c.obj.equals(x)) {
                    ins.remove(c);
                    if (c.i>1) ins.insert(new Coppia(c.obj,c.i-1));
                    return;
                }
            }
        }
        public int molt(Object x) {
        ..... for (Iterator itr =ins.elements(); itr.hasNext ();) {      .....      Coppia c = (Coppia)
itr.next();
        .....           if (c.obj.equals(x))
                            return c.i;
            }
            return 0;
        .....
        }
}
```

```
        }
}
```

c) Dimostrare che il rep invariant RI trovato è effettivamente un invariante per la classe MultiSet;

Giustifichiamo ciascuna delle quattro clausole del RI

- **il costruttore istanzia un oggetto di tipo set**
- **insert e remove inseriscono in ins solo oggetti di tipo coppia**
- **le coppie inserite in ins dal metodo insert hanno i = 1, oppure un i piu' alto di una coppia che era gia' in c.ins; le coppie inserite dal metodo remove hanno un i piu' basso di 1 ma solo se la coppia ha un i > 1, quindi esse hanno un i > 1-1 = 0.**
- **prima di inserire una coppia, insert e remove eliminano la coppia contenente lo stesso oggetto da ins. Ovviamente non potra' quindi mai esserci piu' di una coppia con lo stesso oggetto.**

d) Un programmatore inesperto, pensando di semplificare il codice, implementa la seguente remove

```
public void remove(Object x){
    for (Iterator itr =ins.elements(); itr.hasNext ();) {
        Coppia c = (Coppia) itr.next();
        if (c.obj.equals(x)) {
            ins.remove(c);
            ins.insert(new Coppia(c.obj,c.i-1));
            return;
        }
    }
}
```

Dimostrare che la proprietà RI in questo caso non è più garantita.

Viola la condizione che c.i sia >0

PROVA 2: Esercizio 2.1 (3 punti)
Si consideri il seguente frammento di programma Java:

```
class A …. {
    public synchronized int f (B b) {
        b.g (…);
    }
    …
}

class B …. {
    public synchronized void g (A a) {
        a.f (…);
    }
    …
}
```

Si supponga inoltre che esistano due threads t1 e t2. Il thread t1 contiene la seguente istruzione:

pippo.f (pluto);

Il thread t2 contiene invece la seguente istruzione:

pluto.g (pippo);

dove pippo e pluto sono oggetti delle classi A e B rispettivamente.

Il programma puo` generare un comportamento anomalo (chiamato *deadlock*) in cui il thread t1 blocca al thread t2 l'accesso all'oggetto pippo e, al tempo stesso t2 blocca a t1 l'accesso all'oggetto pluto, cosi` che nessuno dei due thread puo` ulteriormente procedere. Spiegare chiaramente perche` mostrando un esempio possibile di sequenze di eventi a runtime che dia luogo a un deadlock.

Si supponga che
(1) t1 invochi f sull'oggetto riferito da pippo, e immediatamente dopo
(2) t2 invochi g sull'oggetto riferito da pluto.
Siccome (1) effettua un lock sull'oggetto riferito da pippo, il task t2 non puo' procedere ad eseguire l'invocazione del metodo f; analogamente, t1 non puo' procedere ad eseguire l'invocazione del metodo g perche' l'oggetto sul quale operare e' bloccato da (2) che ha effettuato il lock. Pertanto nessuno dei due thread puo' procedere nella computazione.

PROVA 2: Esercizio 2.2 (3 punti)
Parte 1

Si supponga che la specifica di un metodo sia la seguente:

```
int metodoTest( int x) {
        //EFFECTS: se x >= 10 restituisce la cifra meno significativa del numero
        //Se 0<=x<=9 restituisce il valore di x
        //Se x<0 restituisce -1
}
```

Si definisca un insieme di dati di test secondo una strategia "black box", fornendo una sintetica motivazione dei dati proposti.

Si scelgano i valori ai confini dei domini: x=9, x=10, x=0, x=1 e i valori all'interno dei domini: x= 17, x=5, x=-9.

Parte 2

Si supponga di voler testare il metodoTest nell'ipotesi che non contenga cicli, ma solo istruzioni collegate in sequenza ed if-then-else. A tale proposito, si consideri una strategia di test (detta di copertura delle diramazioni) in cui ogni uscita vera e falsa degli if risulta eseguita da almeno un dato di test. Si consideri la seguente affermazione:

Se un metodo non contiene cicli ma solo istruzioni collegate in sequenza e mediante if-then-else allora il criterio di copertura delle diramazioni coincide con il criterio di copertura di tutti i cammini.

E` vera l'affermazione? Se SI, dare una sintetica dimostrazione. Se NO, fornire un controesempio di possibile metodo f.

NO: p. es. il seguente programma:
if (a==1) then B else C
if (b==1) then D else E
per la copertura delle dimazioni bastano due dati di test: p.es. (1,1) e (0,0).
Per la copertura dei cammini occorrono quattro dati (p.es. (1,1) (0,1) (1,0) (0,0))

PROVA 2: Esercizio 2.3 (7 punti)

Si considerino le seguenti specifiche di una classe astratta Figura e di due classi concrete Triangolo e Rettangolo che la estendono:

```
public abstract class Figura {
      public abstract void disegna() //EFFECTS: disegna la figura sullo schermo
}
public class Triangolo extends Figura{
      public Triangolo (float angolo1, angolo2, lato)
      // un triangolo è determinato da due angoli e dalla dimensione del lato compreso.
      public float angolo1();//EFFECTS: restituisce il valore dell'angolo più a sinistra di this
      public float angolo3();//EFFECTS: restituisce il valore dell'angolo più a destra di this
      public float angolo2();//EFFECTS: restituisce il valore dell'angolo intermedio di this
      public void disegna() //EFFECTS: disegna un triangolo sullo schermo
}

public class Rettangolo extends Figura {
      public Rettangolo (float altezza, float base)
      public float base() //EFFECTS: restituisce il valore della base di this
      public float altezza() //EFFECTS: restituisce il valore dell'altezza di this
      public void disegna() //EFFECTS: disegna un rettangolo sullo schermo
}
```

Si desidera ora definire un contenitore di oggetti di tipo Figura, con il vincolo che il contenitore sia omogeneo, nel senso che contenga solo figure "simili"fra loro. Il concetto di somiglianza è stabilito in modo particolare per ogni Figura. Due oggetti di tipo Triangolo sono simili se gli angoli interni corrispondenti sono uguali. Due rettangoli sono simili se hanno lo stesso rapporto base/altezza.
Si desidera utilizzare un approccio "related subtype".

```
interface Verificatore {
//OVERVIEW: tipo immutabile che confronta la somiglianza di due figure… … …
      public boolean simili(Figura f1, Figura f2) throws ClassCastException, NullPointerException;
            //EFFECTS: se f1 e f2 non sono di tipi confrontabili, ClassCastException
            // altrimenti true sse f1 è simile a f2 }
public class NotSimilarException {….}

public class ContenitoreOmogeneo{
// OVERVIEW: Un ContenitoreOmogeneo è un insieme illimitato di oggetti di tipo Figura, tutti
// simili fra loro. Usa un Verificatore per determinare la somiglianza geometrica degli elementi.
      public ContenitoreOmogeneo (Figura f, Verificatore v)          // costruttore
          //EFFECTS: this è inizializzato in modo da contenere la Figura f.
          // v diventa il verificatore da usare per i test di somiglianza successivi.
      public void inserSimile (Figura x) throws NullPointerException, NotSimilarException {…}
            // MODIFIES this
            // EFFECTS se x è null lancia NullPointerException
            // altrimenti se x è geometricamente simile alle figure già inserite inserisce x in this
            // altrimenti lancia NotSimilarException
      public void elimina (Figura x)
            // MODIFIES this
            // EFFECTS se x è null lancia NullPointerException
            // altrimenti elimina la figura equals a x in this
}
```

a) Scrivere la specifica e l'implementazione di classi opportune che consentano l'inserimento e la rimozione di oggetti di tipo Triangolo e di tipo Rettangolo in oggetti di tipo ContenitoreOmogeneo, utilizzando l'approccio Related Subtype.

```
Class TriangVer implements Verificatore {
      public boolean simili(Figura f1, Figura f2) throws ClassCastException, NullPointerException;
            //EFFECTS: se f1 e f2 non sono di tipi confrontabili, ClassCastException
            if (!(f1 instance of Triangolo) || !(f2 instanceof Triangolo)) throw new
ClassCastException();
            Triangolo t1 = (Triangolo) f1;   Triangolo t2 = (Triangolo) f2;
            return (t1.angoloMin () == t2.angoloMin() && t1.angoloMax () == t2.angoloMax());
      }
//basta il costruttore di default
}

Class RettVer implements Verificatore {
      public boolean simili(Figura f1, Figura f2) throws ClassCastException, NullPointerException;
            //EFFECTS: se f1 e f2 non sono di tipi confrontabili, ClassCastException
            if (!(f1 instance of Rettangolo) || !(f2 instanceof Rettangolo)) throw new
ClassCastException();
            Rettangolo r1 = (Rettangolo) f1;   Rettangolo r2 = (Rettangolo) f2;
            return (r1.base/r1.altezza() == r2.base()/r2.altezza());
      }
//basta il costruttore di default
}
```

b) Completare poi il codice seguente (trascurando le dichiarazioni di import):

```
class Prova {
      void main(String args[])
            Figura f1 = new Triangolo(60,30,5);
            Figura f2 = new Rettangolo(10,8);
            ContenitoreOmogeneo c1, c2;
      //inizializzare qui c1 con f1:
      c1 = new ContenitoreOmogeneo(f1, new TriangVer()); .....
      //inizializzare qui c2 con f2:
      c2 = new ContenitoreOmogeneo(f2, new RettVer());.....
```

}

c) Qual è il presumibile effetto dell'esecuzione, come ultima linea del metodo main(), dell'istruzione c1.inserSimile(f2)?

Presumibilmente, il metodo inserSimile di c1 cercherá di chiamare il metodo simili del proprio Verificatore: questo provochera' un ClassCastException. NB: non essendo definita l'implementazione di ContenitoreOmogeneo, altre soluzioni ragionevoli possono essere accettabili, purche' motivate.